UNIVERSITY OF PORTSMOUTH
FACULTY OF TECHNOLOGY

Academic Year 2026/27

Microcontrollers and Programmable Logic M21403

Problem Based Learning CF-1-ORAL-CW 40%

Deadline For Submission:	The deadline is the **START** of the final lab session of the module.
Submission Instructions	During the final lab session, you will be required to demonstrate the operation of the system you have designed. Other aspects of the project may be discussed during the presentation session but all information relating to the project should be uploaded to the logbook on Moodle ***before*** the start of the final lab session. This includes: • State Transition Diagram and Table • Plan for the code • Final source code as a text file so that it can be executed • Test Results It is expected that all group members will attend the final demonstration to answer questions relating to the design and operation of the system as required.
Instructions for completing the assessment:	A logbook of all work on the project should be kept using the logbook on Moodle. There should be a clear link between from the initial design through to the final implementation. All code should be fully commented and presented on the template provided. As a minimum, weekly uploads of a copy and paste of the current version of the code is expected. It should be clear from the logbook what each member of the team has done each week.
Examiners:	Dr Richard Walters

Traffic Light Controller Finite State Machine Project

Basic System Requirements

- You are to design a controller for a traffic light system.
- The traffic lights are to operate according to the UK standard.
    - The traffic light phases are Red, Red-Amber, Green, Amber, Red
- The lights are to control a crossroads which includes a pedestrian crossing.
- There is one major road, one minor road and a pedestrian crossing on all roads.
- The implementation of the lights is a reasonably simple one.
    - Both directions of the main road are green together
    - Both directions of the side road are green together
    - The crossing is on all four roads, and all traffic must be stopped when the crossing is green
- Sensors will be used to detect traffic on both directions of the side road and both directions of the main road. Each of these is in the form of an opto-resistor.
- There are four push button switches for the crossing, each with associated LED to represent the 'wait' light on a real crossing.

Design Constraints

These are restrictions which have been placed on the project by staff for practical or operational reasons.

- The system should be implemented in C, using the PIC18F452.
- All development and testing will need to be completed in simulation on MPLAB X. Once you have developed a solution which you can demonstrate is fully operational in simulation, you will be able to book a session on one of the demonstrator boards to perform a final check that all works correctly. These sessions are subject to availability, are only available during the lab sessions and must be managed by staff.
- When you come to demonstrate the system, you will need to load your code onto one of the demonstrator boards and run it. You will need to think carefully about how you will simulate your code to ensure that all aspects of the code run correctly.

System Block Diagram

There are a number of different ways in which this system could be implemented but one possible implementation is shown in figure 1 overleaf. At the heart of the system is a finite state machine. This will control the operation of the traffic lights and the progression from one 'state' to another. At its most basic, you can think of the states as being the different outputs from the system.

Because the inputs from the road sensors are analogue, you will need to do some input processing on these to convert these analogue signals to digital. Because the state machine will only process based on the inputs *at that time* you may also need to think about whether you need some internal way of remembering the activation of any input sensors until they are responded to. There are four different sensors, one for each road and each of these will need to be processed independently.

The pedestrian crossing buttons are wired in parallel on the board, so pressing any of the buttons will trigger the same crossing input. This is an active low input and is connected to RB0 so can be used to trigger an interrupt on the board. Again, you will need to consider if you need to internally 'remember' that this has been pressed in order to respond later.

The output of the state machine is the number of that state- although you can also consider the output decoding as part of the design of the state machine- so you will need to determine how you go about decoding the various state numbers into the required outputs in that state.

The internal co-processing block has been included because you may wish to include time delays or other processes which aren't actually part of the state machine itself, but which will interact with the state machine to affect its progress.

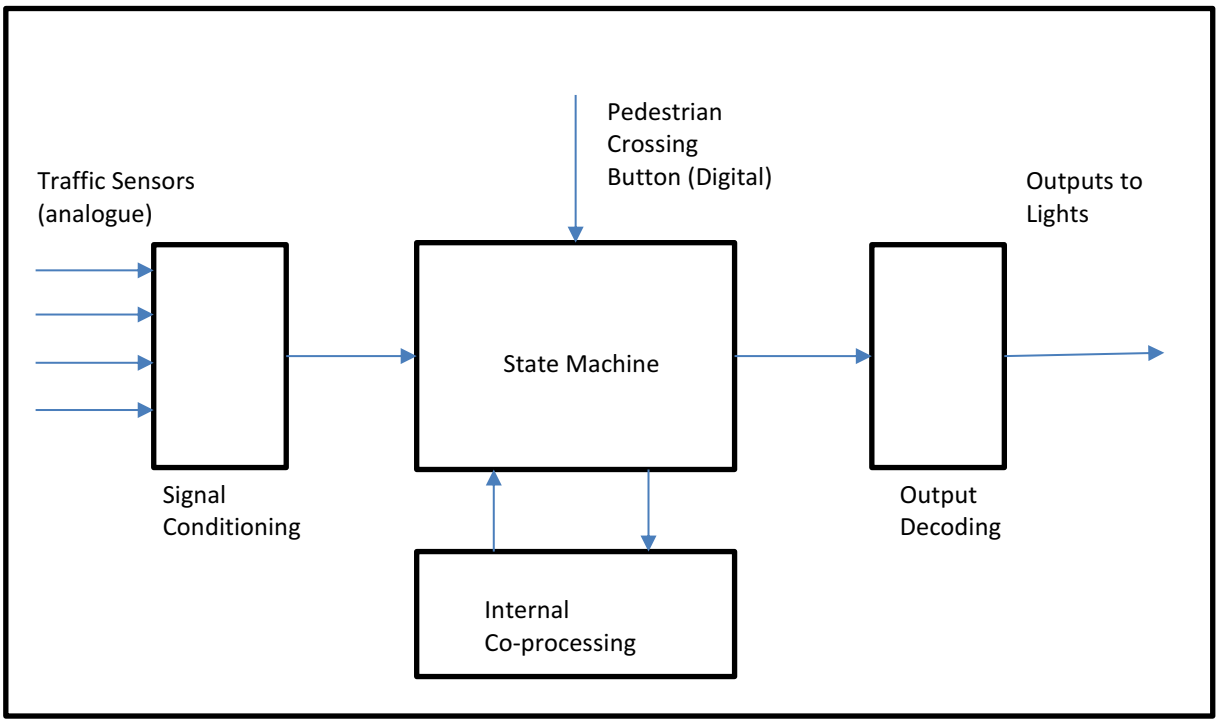


Figure 1: Possible Block Diagram of Traffic Light System

Project Stages

- A functional design of a *finite state machine* to satisfy the requirements above will be produced.
- The state machine should be implemented in C. There should be a clear link between the state machine design and the code developed to implement it.
- The code should then be written and simulated in MPLAB X to confirm that it will work as required on the demonstrator system.
- Tests will be undertaken to demonstrate that the system meets the requirements.
- A review will be undertaken to identify any shortcomings or improvements which could be made.

Although there is only a single formal deadline at the end of the project, in order to allow sufficient time for implementing the design, it is recommended that you have completed the design of the state machine by the week after consolidation week.

Connections to the Demonstration Board

The connections to the demonstration board are set and need to be adhered to in order for the system to work correctly. These are given in the table below.

Port Pin	Purpose		Port Pin	Purpose
RA0	Analogue Input First main road traffic sensor See below for details[1]		RD0, RE0, RB2	Digital Output Pedestrian Crossing Greens These can all be driven with the same signal.
RA1	Analogue Input First side road traffic sensor Operates as RA0		RD1, RE1, RB3	Digital Output Pedestrian Crossing Reds
RA2	Analogue Input Second main road traffic sensor Operates as RA0		RD2	Digital Output Side Road Green
RA3	Analogue Input Second side road traffic sensor Operates as RA0		RD3	Digital Output Side Road Amber
RA4-6	*Not Used*		RD4	Digital Output Side Road Red
RB0	Digital Input Pedestrian crossing buttons- all are connected in parallel to this input[2] Active Low		RD5	Digital Output Main Road Green
RB4- RB7	Digital Outputs[3] Wait lights Each wait light is driven separately. You need to ensure all four are correctly driven		RD6	Digital Output Main Road Amber
			RD7	Digital Output Main Road Red

Note:

[1] The traffic sensors are analogue voltage signals which are generated by potential dividers consisting of an opto resistor and a fixed resistor in series. A voltage greater than about 1.4V indicates the presence of a vehicle. A voltage less than about 0.5V indicates the absence of a vehicle.

[2] There is no external pull-up on the switches, they are just a push switch to ground.

[3] All Outputs are Active High

Suggested Work Breakdown

You are all required to be involved in the initial design of the system. During this phase you should work out, in general terms, what the system is required to do. At this stage you are *not* interested in *how* the system will achieve this, that comes later.

You are required also to be involved in the design and development of the finite state machine which is at the heart of the system, and which is one of the main learning outcomes of the project.

When it comes to coding the system, I would suggest you break the work down into parts and each one work on separate parts of the code, test them, and then bring them together. Of course, in order to be able to do this you will need to be clear on where the boundaries are between the different parts of the code, and in what form the information will be passed from one to the other.

A possible division would be:

- One person works on the input signal conditioning, developing the ADC input code and testing it. Remember that you will need to test it in simulation because you will not have access to the board itself.
- A second person could then work on the development of the state machine code. There are several ways in which this could be done, but one of the simplest is probably to use a *switch case* function in C.
- The person who finishes their part first could then work on the peripheral coding, developing the output decoding and any additional internal processing outside of the state machine itself.

Once you have tested all these parts in isolation, you can then bring them together for final testing and demonstration.

Although I am suggesting you work on separate parts of the code individually, it should be noted, that the whole system is the group's responsibility, so if one member is struggling, others should help. It will not be acceptable to present a non-working system and then blame one or other member of the group.

Keeping a Record of Your Work

It is expected that all work undertaken on the project will be recorded in the logbook in real time- that is to say, at the time you do that work. These logbook entries must show continual development work on the system throughout the project and must show the various parts of the project being completed in the correct order. It is not expected that every upload will contain the full and final version of the state machine, code design or code, and is may be that you will need to go through and revise parts of the design as you identify issues but the first versions of each phase must appear at the correct points in the project.

It is also expected that there will be some sort of detail as to what has been done by the group members in that week, and what each will be doing during the next week.

Testing on the Demo System

All initial testing of the code must be completed on MPLAB X. You should use the facilities there to prove amongst other things:

- That the code runs
- That all states in the state machine are stepped through in the correct order
- That all inputs are responded to correctly
- That all outputs are correct
- That any timing is correct

Only once you have evidenced that these have all been carried out and are passed, to the satisfaction of staff on the module, will you be invited to undertake a test on the demo board which will allow you to check that there aren't any unforeseen or unexpected issues. You are not allowed to use the demo board for ongoing fault-finding and code correction. You will need to consider what tests must be undertaken and what information recorded to prove that the system operates correctly.

Safety

You should not be in a position where traffic and pedestrians have a green light at the same time, nor should both the major and minor roads have a green light at the same time.
To this end, the exact sequence of the lights should be considered, and any potential issues resulting from any changes you might make.

Further considerations

Should you have completed the basic system as detailed above, you may wish to consider the following to further enhance your design:

- Timing differences: Not all phases of a traffic light sequence normally last the same time. How might you implement this?
- Being 'bike-friendly': bicycles are not normally heavy enough to trigger a traffic sensor in the road. If the road is very quiet and only a cyclist is waiting on the side road, or if the sensor fails, how might you allow for traffic from both roads to proceed.
- If traffic on one of both roads is very heavy, causing repeated triggering of the sensor on a road, how will you ensure that traffic on the other road is not permanently held?

There may be other design enhancements you wish to consider. All should be justified in terms of the benefits their inclusion would bring to the system.
Submissions for all parts of the project should be made to the appropriate places on Moodle.
You should gain sign-off on the basic design before proceeding to add additional features.

Indicative Grade Specifiers

Aspect	Mark
State Machine Design This aspect will look at the development of the state transition diagram, and the state transition table, is it valid, is it complete etc.	10
Initial Code Design This section will look at what evidence is presented, of the design of the code. You should note that design is expected to be completed in advance of any coding being undertaken.	5
Final Code Does the code match the design? Is there a clear link to the start machine design? Is the code well structured? Is the code well commented?	10
Operation This will be undertaking in the final lab session, and will look to see the system in operation	5
Results and Conclusions Any results uploaded which show how the system has been tested. Valid conclusions drawn from those results	5
Additional marks available for additional features, particularly elegant solutions or other credit-worthy inclusions	5
Contribution It is expected that all students work equally on the project. It is not necessary that group members repeat the work of others in the group, but consistent levels of work should be undertaken. Should this not be the case, a sliding scale will be used to moderate the marks awarded to individual group members, such that the total contribution will always be x2. Note that non-attendance at the final submission session will always result in a mark of zero.	x0.5 – 1.5
Total	**40**